NMC-OPT STUDIO: INSTALLATION, CONFIGURATION & EXECUTION GUIDE

Complete Technical Operations Manual for Local Deployment and CLI Pipeline Execution

Target Audience: Software Engineers, Academic Evaluators, and Compiler Researchers
Repository URL: https://github.com/MSN-1307/NMC-OPT-CD | **Platform:** Windows / Linux / macOS
Prerequisites: Python 3.10+, Node.js 18+, C Compiler (gcc/clang), and Git

1. SYSTEM PREREQUISITES

Before initializing NMC-Opt Studio, verify that the following runtime environments are installed:

- **Python:** Version 3.10 or higher (Python 3.11, 3.12, 3.13, and 3.14 verified). Ensure pip is updated.
- **Node.js:** Version 18.0 or higher with npm (Node.js 20+ LTS recommended).
- **C Compiler:** GCC (MinGW for Windows, GNU GCC for Linux) or Clang for native C kernel compilation audits.
- **Git:** Version 2.30+ for repository cloning and version control.

2. STEP 1: REPOSITORY CLONING & DIRECTORY INSPECTION

Clone the remote Git repository and navigate into the root project directory:

```
git clone https://github.com/MSN-1307/NMC-OPT-CD.git cd NMC-OPT-CD
```

3. STEP 2: BACKEND CONFIGURATION & SERVER LAUNCH

The backend is built with FastAPI and runs the analytical cost modeling, pycparser AST profiling, and SSA IR generation passes.

```
cd backend python -m venv venv # Windows PowerShell: .\venv\Scripts\Activate.ps1 |
macOS/Linux: source venv/bin/activate pip install -r requirements.txt python -m uvicorn
app.main:app --host 127.0.0.1 --port 8008 --reload
```

Verification Endpoints: Swagger Docs at http://localhost:8008/docs | Health Check at http://localhost:8008/api/health

4. STEP 3: FRONTEND INSTALLATION & VITE DEVELOPMENT SERVER

In a separate terminal, initialize the React / TypeScript user interface:

```
cd frontend npm install npm run dev
```

Open your web browser and navigate to: **http://localhost:5173**

5. STEP 4: STANDALONE COMMAND-LINE INTERFACE (CLI) EXECUTION

NMC-Opt Studio includes a standalone CLI utility (backend/cli.py) enabling direct terminal evaluation without opening a browser:

```
# Evaluate Vector Addition with 2,000,000 elements python backend/cli.py --benchmark
vector_add --size 2000000 # Evaluate Matrix-Vector Multiplication (2048 x 2048 matrix)
python backend/cli.py --benchmark gemv --size 4194304 --rows 2048 --cols 2048 # Evaluate
arbitrary custom C loop kernel file python backend/cli.py --custom path/to/my_kernel.c --
size 1000000
```

6. STEP 5: AUTOMATED TESTING & BUILD VERIFICATION

Verify algorithmic correctness across compiler modules using the automated pytest test suite:

```
python -m pytest backend/tests -v cd frontend && npm run build
```

7. TROUBLESHOOTING & COMMON RESOLUTIONS

- **Port Conflict on 8008:** If port 8008 is in use, modify the uvicorn start command: `python -m uvicorn app.main:app --port 8010` and update frontend API baseUrl.
- **PowerShell Execution Policy:** If Activate.ps1 cannot be loaded, run: `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`.
- **pycparser Syntax Errors:** Ensure uploaded custom C code includes standard function wrappers and avoids compiler-specific assembly extensions.